Leveraging Design Rules to Improve Software Architecture Recovery

Yuanfang Cai
Dept. of Computer Science
Drexel University
Philadelphia, PA, USA
yfcai@cs.drexel.edu

Hanfei Wang
State Key Laboratory for Novel Software Technology
Dept. of Computer Science and Technology
Nanjing University, Nanjing, 210046, China
wanghanfei1988@seg.nju.edu.cn

Sunny Wong
Clinical Solutions R&D
Siemens Healthcare
Malvern, PA, USA
sunny.wong@siemens.com

Linzhang Wang
State Key Laboratory for Novel Software Technology
Dept. of Computer Science and Technology
Nanjing University, Nanjing, 210046, China
lzwang@nju.edu.cn

ABSTRACT

In order to recover software architecture, various clustering techniques have been created to automatically partition a software system into meaningful subsystems. While these techniques have demonstrated their effectiveness, we observe that a key feature within most software systems has not been fully exploited: most well-designed systems follow strong architectural *design rules* that split the overall system into modules. These design rules are often manifested as special program constructs, such as shared data structures or abstract interfaces, which should not belong to any of the subordinate modules. We contribute a new perspective of architecture recovery based on this rationale, which enables the combination of design-rule-based clustering with other clustering techniques, as well as enabling the splitting of a large system into subsystems. We evaluated our approach both quantitatively and qualitatively, using both open source and real industrial software projects.

Categories and Subject Descriptors

D.2.11 [**Software Engineering**]: Software Architectures

General Terms

Design

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.
QoSA'13, June 17–21, 2013, Vancouver, BC, Canada.
Copyright 2013 ACM 978-1-4503-2126-6/13/06 ...$15.00.

Keywords

Architecture Recovery, Design Structure Matrix

1. INTRODUCTION

In order to recover high-level architectural structure, various clustering techniques have been extensively studied [3, 13, 18, 23, 26]. These techniques aggregate program entities, such as procedures and classes, into modules based on different rationales, including coupling and cohesion [3, 13, 16], naming patterns [30], and minimal information loss [1]. These techniques are usually evaluated and compared using large-scale legacy systems [18] and often generate drastically different clusterings [25].

We observe that an important feature that exists in many software systems has not been fully exploited—that is, the existence of *design rules*. According to Baldwin and Clark [4], *design rules* are defined as stable design decisions that decouple the rest of the system into *modules*. In his seminal paper, Parnas [21] proposed the cornerstone concept of *information hiding*; the essence is to decouple a system into modules by creating stable interfaces, such as abstract data types, that hide and decouple the "secrets" into modules. Baldwin and Clark [4] formalized the concept of such stable interfaces as *design rules*, and formalized the action of "decomposing a system into modules" as a *splitting* operation.

Well-designed software systems often have strong design rules, manifested as key header files, globally shared data structures, abstract interfaces in object-oriented systems, etc. If a system applies architectural patterns or design patterns, such as the model-view-controller pattern or abstract factory pattern, there must be key interfaces or abstract classes leading the patterns, which in essence are instances of design rules. For example, applying the abstract factory pattern [10] requires creating an *abstract factory* interface that decouples multiple concrete factories. In this case, this abstract factory interface is one *instance* of a design rule.

The problem is that, by using traditional software clustering rationales (e.g., coupling and cohesion), the strong characteristics of design rules cannot be fully exploited and the recovered design structure may not reflect their architectural significance nor their splitting effects. Design rules in a system should not belong to any modules; instead, they dominate subordinate modules and frame the overall modular structure. Using prevailing clustering techniques, however, these design rules are often aggregated into other modules. As a result, their dominating role in the architecture is not revealed.

In this paper, we contribute a new perspective of architecture recovery: an effective recovery approach should first recover the backbones, that is, the design rules, within the system, and then recover the modules created by these design rules. Based on this rationale, we propose an algorithm, which we call *ArchDRH*, that can be used alone or be combined with other clustering methods, such as ACDC [30] or Bunch [16], to recover software architectures more effectively. Derived from our prior work [32], the ArchDRH algorithm first clusters the components of a codebase into a hierarchy—aggregating the design rules to the top layers of the hierarchy, manifesting their dominating position and their independence from other subsystems. The subsequent layers of the hierarchy contain mutually independent modules that depend on design rules in the upper layers only.

Within each module generated by the ArchDRH algorithm, the user can plug in other clustering algorithms, depending upon the system under analysis. For example, some systems may follow strong naming convention. In this case, applying ACDC after ArchDRH may recover the architecture more accurately. Our ArchDRH tools are configurable to support such variations. We also contribute a method to extract a subsystem that contains only modules that follow the specified set of architectural design rules. Using this technique, the user can conduct various analysis techniques, for example, to check how many and which design patterns a function module participates in.

In this paper, we introduce the following ArchDRH family members: (1) *ArchDRH-Re*, the algorithm that applies ArchDRH recursively within modules, (2) *ArchDRH-ACDC*, the algorithm that applies ACDC clustering within ArchDRH decoupled modules, (3) *ArchDRH-Bunch*, the algorithm that applies Bunch clustering within ArchDRH decoupled modules, and (4) ArchDRH-Split, the algorithm that extracts a subset of the system based on a set of selected design rules. This family is extensible to integrate with other clustering methods, such as the techniques presented by Andritsos and Tzerpos [1], and Eisenbarth et al. [8], which is part of our future work.

We evaluate the ArchDRH family both quantitatively and qualitatively using software projects of different sizes and domains, including both open source and real industrial software projects. For quantitative analysis, we use the structural indicators proposed by Shterm and Tzerpos [25] to compare the clusterings produced by Bunch, ACDC, ArchDRH-Re, ArchDRH-ACDC, and ArchDRH-Bunch. For qualitative evaluation, we either present the ArchDRH-based clustering to our collaborators to get their opinion, or compare the clustering results with existing documentation.

The evaluation results demonstrate that ArchDRH can effectively improve existing architecture recovery techniques. More interestingly, the studies demonstrate the possibility of identifying architectural drift caused by implementation errors, which are manifested as unexpected dependencies to design rules. The rest of this paper is organized as follows: Section 2 explains how our approach differs from other related work. Section 3 illustrates the ArchDRH family of clusterings using a running example. Section 4 introduces the ArchDRH clustering algorithms. Section 5 presents our evaluation results. Section 6 discusses threats to validity and future work. Section 7 concludes.

2. RELATED WORK

In this section, we compare our work with traditional clustering techniques, representative architecture and design pattern recovery techniques, and our prior work.

Traditional Clustering Techniques

Based on the principle of low coupling and high cohesion, various clustering algorithms have been proposed [3, 13, 16–18, 23, 26]. For example, the single-linkage algorithm (SLA) [13] and complete linkage algorithm (CLA) [3] are used to calculate distances between clusters based on their similarity—that is, whether they access the same set of global variables. Viewing software clustering as an optimization problem, researchers have used genetic algorithms to decompose a system to modules, such as the work of Harman et al. [12] and Mancoridis et al. [20]. Bunch [20] is a representative tool that uses a genetic algorithm to conduct clustering-based architecture recovery, aiming to optimize coupling-and-cohesion-based quality measures.

Researchers have also used information analysis techniques to improve clustering methods, such as concept analysis [8, 27], latent semantic analysis (LSA) [15], concern analysis [1], patterns (ACDC) [30] and data mining techniques [2, 7, 24]. As a representative tool, ACDC [30] first clusters a system based on a number of subsystem patterns, such as directory structure pattern and body-header pattern, and then uses orphan adoption methods to aggregate leftover elements.

As aforementioned, these techniques do not take the existence of design rules and resulting modules into consideration. Our approach combines design rule clustering with these techniques to reap the best.

Architecture Recovery and Design Pattern Identification

Tsantalis et al. [28,29], among others, have proposed methods to identify design patterns in code. Guéhéneuc and Antoniol [11] proposed a multi-layered approach to detect design motifs. By contrast, our technique leverages the existence of design patterns to improve the accuracy of clustering. We leverage the interfaces or classes that lead a pattern, discovered by other design pattern detection tools, as the basis to decompose a system into smaller subsystems.

Design Rule Hierarchy

In our prior work [32], we proposed the concept of *design rule hierarchy* (DRH) for the purpose of maximizing task parallelism. This original DRH algorithm was not designed for the purpose of architecture recovery, and often generates very large or very small modules. The ArchDRH clustering method presented in this paper is derived from the original DRH algorithm, but is significantly revised for the purpose of architecture recovery. The differences between the original DRH and the ArchDRH clusterings are elaborated in Section 3 and Section 4.

3. ILLUSTRATIVE EXAMPLES

In this section, we use a running example first to introduce background concepts, the *design structure matrix* (DSM) [4] and *design rule hierarchy* [32]. We also use this example to illustrate the key difference between the ArchDRH clustering family and two other prevailing clustering techniques: Bunch [16] and ACDC [30].

The example we use is an implementation of a maze game, whose design is slightly modified from the classic design pattern book [10]. A maze consists of multiple rooms, each with walls and a door to another room. The game is designed to support two variations by applying an abstract factory pattern. The source code is written in Java, and has 15 classes. Since the implementation has been confirmed to be correct, we expect that an effective clustering technique should be able to faithfully recover the designed structure by revealing the existence of two separated concrete factory modules, the blue maze module and the red maze module.

3.1 Background Concepts

Design Structure Matrix (DSM)

In this paper, we represent the modular structure of a software system, reverse-engineered from code, using DSMs [4]. A DSM is a square matrix; its columns and rows are labeled with the same sequence of elements in the same order. If an element at row x depends on the element in column y, then the cell (rx, cy) is marked. Figure 1 depicts a DSM reverse-engineered from the maze game implementation. This DSM lists the 15 classes in three layers (the box with solid border) and five modules (the inner blocks along the diagonal with light grey background), showing a hierarchical structure. The dependencies in the DSM model the syntactical relation between classes and interfaces, such as method call or inheritance relation.



Figure 1: Maze Game DSM Generated by ArchDRH-Re and ArchDRH-ACDC

We use our tool, Titan [31], to visualize DSMs. Titan accepts a *.dsm* file, which only represents the dependency relation between elements, and a *.clsx* file that represents the clustering (decomposition) of these elements. A hierarchical structure is represented as a tree structure. In Titan, the user can collapse or expand the tree nodes, and view the corresponding DSM. All the DSMs in this paper are exported from Titan. When comparing multiple clustering methods, we transform the output of these clustering tools into *.clsx* files, and these files share the same *.dsm* file because they all work on the same dependency relation. As a result, we are able to use Titan to view DSMs clustered by different software clustering tools. Figure 2b depicts the DSM transformed from the output of Bunch. Figure 2c depicts the DSM transformed from the output of ACDC.

Design Rule Hierarchy (DRH)

In our prior work [32], we proposed the concept of a *design rule hierarchy* that has two key characteristics: (1) the upper layers of the hierarchy contain design rules that decouple the rest of the system, and the subsequent layers contain design decisions that only depend on decisions in upper layers; (2) modules within a layer are *mutually independent* from each other, that is, there are no dependencies between the modules within a layer, which is the most unique feature of DRH. Consequently, each module within a layer can be implemented concurrently and in parallel. The modular structure generated by all the ArchDRH family members share the same characteristics, as shown in the DSMs of Figure 1 and Figure 2a.

ArchDRH vs. DRH

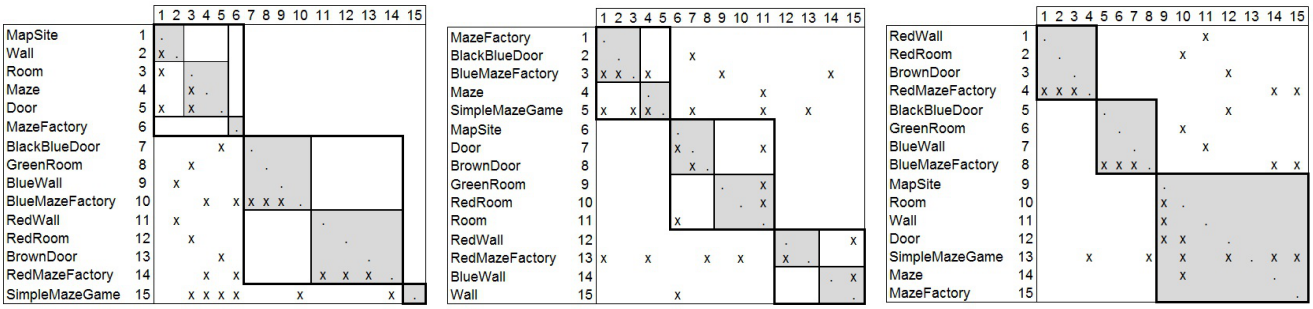
ArchDRH presented in this paper and DRH in our prior work [32] first differ in terms of their approach of calculating *modules*. In ArchDRH, a module can contain elements implementing the same concept or features, in the form of depending on the same set of design rules, but not necessarily having dependencies among themselves. While in the original DRH algorithm, elements are highly coupled. Second, ArchDRH identifies *control modules*, that is, modules that depend on many other elements, but have no dependents. The `SimpleMazeGame` class is a sample control module. This class contains the `main` function, and should not belong to any other module.

ArchDRH and DRH are also different in terms of the basic elements they operate upon, and the dependency structure they manipulate. ArchDRH clustering uses classes, interfaces, or other program constructs as atomic elements, similar as other prevailing clustering techniques. By contrast, our original DRH work uses *design decisions* as basic elements for the purpose of maximizing task parallelism. For example, in DRH, a class was modeled using at least two decisions: a (visible) interface decision and a (hidden) implementation decision. Another key difference is that the original DRH uses dependencies derived from an *augmented constraint network* [6], which captures more indirect dependencies that are not used in ArchDRH.

Consequently, their resulting modular structures are quite different. As reported in our prior DRH paper [32], the DRH-clustered DSM for the same maze game example has 4 layers, 19 modules, and 24 decision variables. By contrast, Figure 1 in this paper depicts the ArchDRH-clustered maze game DSM only has 3 layers, 5 modules and 15 elements.

3.2 ArchDRH Clustering Family

Leveraging the ArchDRH algorithm that manifests design rules and modules, we contribute an ArchDRH clustering family. For each member of this family, the code is first clustered using the ArchDRH algorithm to be introduced in Section 4. Each resulting module can then be further clustered using another clustering algorithm, such as Bunch, ACDC, or additional passes of ArchDRH. In this paper, we present the following ArchDRH family members: ArchDRH followed by one or more passes of ArchDRH clustering recursively (ArchDRH-Re), ArchDRH followed by Bunch clus-



(a) ArchDRH-Bunch DSM (b) Bunch DSM (c) ACDC DSM

Figure 2: Maze Game Implementation Clustered in Three Ways

tering (ArchDRH-Bunch), and ArchDRH followed by ACDC clustering (ArchDRH-ACDC). The ArchDRH family can be extended with new members by combining ArchDRH clustering with other clustering techniques, such as CLA or SLA. Based on the hierarchy generated by ArchDRH, we can also split the overall system into smaller subsystems. We call this splitting algorithm ArchDRH-Split, a special ArchDRH family member. Next, we use the maze game example to illustrate the feature of these ArchDRH members and compare them with the original Bunch and ACDC clusterings.

Figure 1 depicts a DSM reverse-engineered from the maze game implementation and clustered using ArchDRH-Re. In this DSM, class `MapSite` and its subclasses, `Wall`, `Door`, and `Room` form the first module (r1-5, c1-5). The interface `MazeFactory` takes the role of the *abstract factory* in the pattern and is also listed in the top layer. The DSM also shows that these top layer classes (design rules) decouple the elements in the second layer into two separate modules: one module (r7-10, c7-10) contains the classes implementing a blue factory and the other module (r11-14, c11-14) contains the classes implementing a red factory. The bottom layer only contains a single-element module, `SimpleMazeGame`. This class has the static `main()` function that depends on many other classes.

We observe that this clustering faithfully reflects the designed modular structure: the two concrete factory modules are completely separated by the abstract interface and the base classes. The class containing the `main` function acts as a controller class and should not belong to any other module. *ArchDRH-ACDC* generates exactly the same clustering as *ArchDRH-Re*, meaning that applying ACDC within each inner module decoupled by first running *ArchDRH* cannot split these modules further. The clustering produced by *ArchDRH-Bunch* (Figure 2a) is slightly different in that the first module is further split into two inner modules.

Now we compare the ArchDRH family members with the original Bunch (Figure 2b) and ACDC (Figure 2c) clusterings. Figure 2c shows that ACDC generates a similar clustering with ArchDRH-Re, except that that the control class containing the `main` function was aggregated with design rules. Another difference is that ACDC does not generate a hierarchical structure. Figure 2b shows that Bunch clusters the system quite differently. Although the Bunch clustering optimizes coupling-and-cohesion quality measures, it is hard to find the intended modular structure. For example, the module containing `SimpleMazeGame` and `Maze` does not represent any meaningful module of the system.

Given these comparative analyses, it is reasonable to consider the DSM shown in Figure 1 to be the authoritative clustering of this maze game implementation. In the meanwhile, we observe that running ACDC and Bunch alone can generate drastically differen clusterings, consistent with existing research results on clustering comparison [33].

For this particular example, ACDC produces a clustering that is similar to the authoritative one because the source code also demonstrates strong naming pattern that matches ACDC's clustering rationale. But ACDC does not separate the control class that should not belong to any functional module. Clusters generated by Bunch are less useful because coupling and cohesion are not the dominating design rationale of the maze game system.

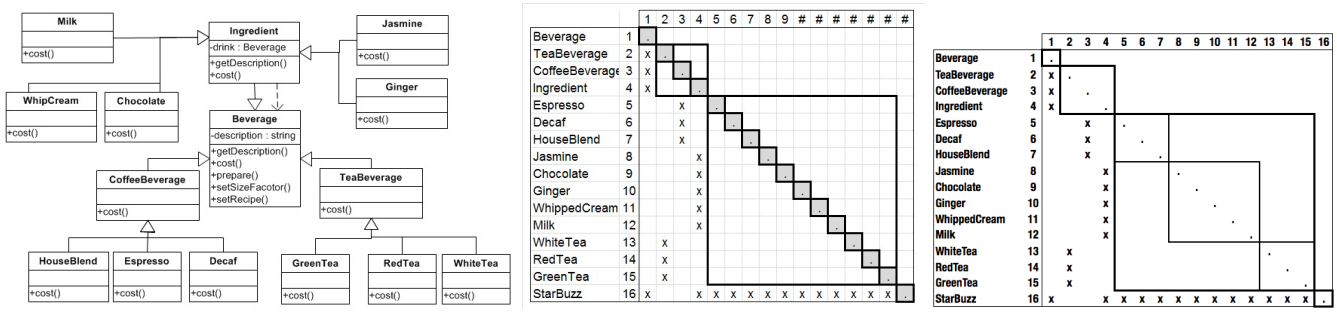
We also observe that ArchDRH-Re, ArchDRH-Bunch and ArchDRH-ACDC generate very similar modular structures. The drastic differences between ACDC and Bunch clusterings are minimized because they are both framed within the same design rule hierarchy.

ArchDRH-Split is a special ArchDRH family member that decomposes a big system into smaller subsystems, using specific design rules as an input. For example, applying ArchDRH-Split on the DSM shown in Figure 1, using `MazeFactory` as the input, we can get a smaller DSM containing only elements 6-15. This sub-DSM (not shown for the sake of space) has only one top-layer design rule, `MazeFactory`, followed by two concrete factory modules in the second layer.

4. ARCHDRH CLUSTERING

In this section, we introduce the ArchDRH clustering algorithm that forms the basis of the ArchDRH family. As with other code clustering techniques, such as ACDC and Bunch, ArchDRH clustering algorithm takes a dependency graph as input, which can be derived from code using a reverse engineering tool. The vertices of a dependency graph model program elements, and edges model their dependency relation, such as function call and inheritance.

We use an example to illustrate the ArchDRH algorithm. Figure 3a depicts the UML model of a coffee/tea ordering system applying a decorator pattern, an example modified from a popular design pattern book [9]. In this design, the `Beverage` class takes the role of *component*, the `Ingredient` class leads the role of *decorator*, and the `CoffeeBeverage` and `TeaBeverage` are concrete components.



(a) Decorator Pattern UML (b) DSM after Step 1 (c) Decorator Pattern DSM

Figure 3: The Design and DSMs of the Coffee Ordering Program

Step 1: Apply DRH

The purpose of this step is to identify design rules and frame the basic hierarchical structure using the original DRH algorithm. As introduced in our prior work [32], the first step is to take the dependency graph as input, and compute its *condensation graph*, in which each vertex represents a strongly-connected component, that is, the classes or functions that are closely coupled by cyclical dependency. A condensation graph is a *directed acyclic graph* (DAG) that contains a partial ordering of the original directed graph. The minimal elements of such a DAG model basic functions. For each minimal element, we aggregate all the vertices along the paths it belongs to, and form a subgraph. After that, we calculate which vertices are shared by these functional subgraphs. These shared elements are candidates of design rules, and are allocated into higher layers of the hierarchy. We then calculate connected subgraphs within each layer to identify mutually independent modules.

Figure 3b depicts the DSM generated after this step. In this DSM, each of the five ingredient classes becomes five independent single-element modules. This DSM follows the two DRH characteristics: elements in lower layers depend on upper layer design rules and modules within a layer are mutually independent. However, it contains many small modules that do not reflect the designed architecture structure. For example, it is not reasonable to make each ingredient class into a module.

Step 2: Identify Conceptual Modules

Although the modules identified within each layer are mutually independent from each other, it is possible that some of these modules follow the same concept and should be aggregated into one conceptual module. For example, although the `Ingredient` interface separates the five concrete ingredients, these ingredients can be considered as a *Ingredient* module since they all follow the same `Ingredient` design rule. The second step of the ArchDRH clustering algorithm first scans all the modules within each layer to see if any of them directly depend on exactly the same set of design rules. If so, the algorithm aggregates these modules into one conceptual module. Figure 3c depicts the DSM after this step. The modules (r5-7, c5-7), (r8-12, c8-12), and (r13-15, c13-15) model the existence of *coffee beverage*, *tea beverage* and *ingredients* modules. The DSM with these conceptual modules maintains the two, previouly described, key characteristics of the original DRH.

Step 3: ArchDRH-Family

If an element has many incoming and outgoing dependencies, it can easily form a large module. We propose the following ArchDRH family members that vary in the last step of ArchDRH clustering to further decompose large modules with interconnected elements.

ArchDRH-Re. ArchDRH-Re algorithm runs the first two steps of the ArchDRH clustering recursively to separate *control modules*. A *control module* is defined as a module that does not have dependents, but depends on a majority of other elements within a layer. Since control classes are usually the minimal elements of the DAG generated in step 1, each round of the recursive function separates the minimal elements into a separate module within a separate layer, and applies ArchDRH again on the rest of the graph. Consider the maze game example. After the first run of ArchDRH, there is only one layer with one module because the control class, `SimpleMazeGame`, connects all other elements.

ArchDRH-Re separates `SimpleMazeGame` into a bottom layer and processes the rest of the graph. The second round of ArchDRH-Re finds two minimal elements, `BlueMazeFactory` and `RedMazeFactory`, which lead to two function modules and two design rule modules. ArchDRH-Re again tries to separate the minimal elements in each of the four new modules, but no new modules can be found because it reaches the stop condition of the recursive function. ArchDRH-Re stops when the density of a module is too low or too high. Very high density suggests that the elements in this module are highly cohesive with each other and they should be clustered into a module. Very low density suggests that the information in this graph is so little that further clustering cannot produce meaningful modules. In the current implementation, for module m with $|E|$ dependencies among $|V|$ classes, we define the stop condition as: $|E| < |V|$ (too sparse) or $|E| > |V|^{1.5}$ (too dense).

ArchDRH-[3rd]. In addition to using ArchDRH-Re to separate control elements and further decompose, we can also use other 3rd-party clustering techniques to decompose a module, according to specific design rationale of the project. In this paper, we explore ArchDRH-Bunch and ArchDRH-ACDC. The user can choose to run ArchDRH once, then apply a 3rd-party algorithm, or to run ArchDRH-Re first to segregate control elements and then apply other algorithms. The user can also choose to run ArchDRH-Re multiple times till it reaches a stop condition, or configure the tool to run one or multiple recursions before applying

a 3rd-party algorithm. For example, the DSM in Figure 2a (ArchDRH-Bunch) is produced by applying ArchDRH-re for two rounds and then applying Bunch within each module.

ArchDRH-Split. It is normal for a function module to follow multiple design rules. For xample, a class may participate in multiple design patterns by implementing different interfaces. In this case, it can be difficult to understand the modular structure framed by these design rules in a large system. We contribute an ArchDRH-Split algorithm to produce a partial view of the overall system based on a specified set of design rules. This algorithm takes as input a set of specified design rules and a DSM clustered by one of the ArchDRH family members. ArchDRH-Split extracts a subset of the DSM in which the specified design rules appear at the top of the DSM, followed by the function modules that depend on the specified design rules. All other elements that do not follow the given design rules are filtered out. Figure 4 depicts a DSM reverse-engineered from JHotDraw, clustered using ArchDRH-Re and decomposed by ArchDRH-Split. This DSM contains all the function modules that follow the `Tool` design rule. We discuss more about ArchDRH-Split in Section 5.

5. EVALUATION

To evaluate the effectiveness of our approach, we compare the ArchDRH family with Bunch and ACDC, two representative clustering techniques.

5.1 Subjects

Table 1 lists the basic information of the eight subject systems we use. The first three subjects are software projects built at Drexel. Subjects 4-6 are the three versions of a real industrial project for which we keep their name anonymous. Subject 7 is an open source project and Subject 8 is provided by our academic collaborators. Subject 4-6 are developed using C# and all others are Java projects.

Table 1: Subject Projects

	Subject	LOC	Classes	Eval
1	Moka	3280	66	Quant.
2	Titan	2032	35	Quant.
3	Minos	1697	22	Quant.
4	Sprint1	18.5K	391	Quant./Qual.
5	Sprint2	25K	857	Quant./Qual.
6	Sprint3	25K	861	Quant./Qual.
7	JHotDraw 5.2	5290	171	Qual.
8	CourseQ&A	3873	52	Qual.

We chose *Moka*, *Minos*, and *Titan*[1] because these systems are designed in house, used to generate DSMs presented in this paper and thus become an interesting self-evaluation subject. *Moka* [31] is a reverse-engineering tool that extracts a UML model from compiled Java code. *Minos* [31] is a program that processes *augmented constraint networks* (ACNs) and generates dependency relation. *Titan* [31] is a DSM tool that allows the user to manipulate the hierarchical structure, visualize DSMs in a scalable way, and export a DSM to a spreadsheet.

Sprint1, Sprint2, and Sprint3 are three releases of an industrial project, which we refer to as Sprint. This project has gone through five years of evolution. Sprint2 evolved from Sprint1 by adding new features. In Sprint3, the architecture was refactored to ease future maintenance. *JHotDraw*[2] (JHD) is a Java framework for creating graphs. We chose this subject because it is well-documented [22] and widely studied [29]. *CourseQ&A* is part of a web-based course management system developed in a university for teachers to create exams and for students to take exams. Table 1 also shows which types of evaluation were applied to these subjects, quantitative and/or qualitative.

Unlike other ArchDRH family members that can be quantitatively compared against ACDC and Bunch, there is no counterpart of ArchDRH-Split to compare with. As a result, we only evaluate ArchDRH-Split qualitatively. We ran our experiments on a Windows PC with 2.30GHz AMD Turion(tm) II P530 dual-Core processor and 4GB of RAM. The longest time used to run an ArchDRH algorithm was 13 seconds.

5.2 Quantitative Evaluation

Following the existing work of clustering technique comparison [14, 18, 25], we need the following elements to conduct quantitative comparison: an *authoritative clustering* for each system, and a comparison metric. The authoritative clustering is supposed to accurately reflect the designer's understanding of the architecture, but we understand that there is no guarantee that the authoritative clusterings provided by our collaborators are accurate. We discuss this threat to validity in Section 6.

Authoritative Clustering Generation

We chose the first 6 subjects for quantitative evaluation because we were able to get their authoritative clusterings. For the three in-house software systems, Moka, Minos and Titan, the third author of this paper used their namespace structure as the authoritative clusterings. Since these systems are small (with 66, 35, and 22 classes respectively), their namespace structures effectively reflect their function modules.[3] For the three versions of Sprint, we asked our industrial collaborators to create authoritative clusterings using Titan, and provide us the *.clsx* files. This was conducted within their company completely independently, without any participation of the authors of this paper.

Intuitively, since ArchDRH always puts design rules into their own modules and layers, separated from function modules, none of these authoritative clusterings would favor the ArchDRH family because neither the authors nor our industrial collaborators intentionally identify or separate design rules in their authoritative clusterings.

Comparison Metrics

We chose the *structure indicators* proposed by Shtern and Tzerpos [25] to quantitatively compare the clusterings generated by Bunch, ACDC, ArchDRH-Re, ArchDRH-Bunch, and ArchDRH-ACDC, against the authoritative clusterings. We chose this evaluation method because they address the issues of other prevailing evaluation methods. More discussion on this topic follows in Section 6.

Following the definition of Shtern and Tzerpos [25], we refer to the five clustering methods as *test* clusterings. If a module, T_i, in a test clustering contains more than 50% of

[1]Moka, Minos and Titan can be downloaded from `http://rise.cs.drexel.edu/projects/`

[2]`http://www.jhotdraw.org/`

[3]The authoritative clusterings for these systems are available at `http://www.cs.drexel.edu/~yfcai/archdrh/data`

the elements of a module, A_i, in the authoritative clustering, then T_i is defined to be a *segment* of A_i. The structure indicator consists of three values:

— *Extraneous Clustering Indicator* (E) measures the number of modules in the test clustering that are not segments of any modules in the authoritative clustering, i.e. the number of modules in the test clustering that are not meaningful.

— *Lost Information Indicator* (L) measures the number of modules in the authoritative clustering that do not have any segments in the test clustering. That is, the number of authoritative modules that are not recognized.

— *Fragmentation Indicator* (F) measures the average number of segments for all authoritative modules recognized in the test clustering. The larger the value of F, the more segments an authoritative module has, meaning that an authoritative module is separated into more parts.

If a test clustering matches exactly the authoritative clustering, then it has an ELF vector of $(0, 0, 1)$. The closer a EFL vector to $(0, 0, 1)$, the more similar the two clusterings.

Evaluation Results.

For each of the 6 subjects, we generate 5 test clusterings using ACDC, Bunch, ArchDRH-Re, ArchDRH-Bunch and ArchDRH-ACDC respectively. For ArchDRH-Bunch and ArchDRH-ACDC, we first run ArchDRH-Re for two recursions to separate control classes, and then run Bunch/ACDC within inner modules. The comparison results are listed in Tables 2, 3, 4, and 5.

Table 2 lists the number of modules in the authoritative decomposition and in the clusterings generated by the five techniques. The table shows that ACDC tends to generate fewer modules than other clustering methods. Table 3, 4 and 5 present the indicator values of the five clustering methods. In these tables, DR, DRA and DRB mean ArchDRH-Re, ArchDRH-ACDC, and Arch-Bunch respectively, and cells containing the best result of the row are indicated using gray background.

For example, in Table 3, the first cell is 67%, meaning that of all the 15 modules generated by ACDC from Moka, 10 of them are extraneous, i.e., do not form meaningful modules. In Table 4, the first cell is 15(20), meaning that of all the 20 modules in Moka authoritative clustering, 15 of them were not recognized by ACDC. In Table 5, the first cell is 1.000, meaning that on average, each authoritative module, if recognized, has one segment in ACDC clustering. Overall, ACDC clustering does not appear to be effective for Moka even though it has the best F value. We make the following observations from these tables.

Which method produces fewest useless modules? Table 3 shows that ArchDRH-Re generates fewest extraneous modules for all subjects except Minos, for which ArchDRH-Bunch and ArchDRH-ACDC are winners. For the three Sprint versions, Bunch generates twice as many extraneous modules as ArchDRH-Re, even though they produce similar number of modules.

Table 2: Module Count

Subject	Auth.	ACDC	Bunch	DR	DRB	DRA
Moka	20	15	21	25	29	27
Titan	13	3	22	15	15	15
Minos	9	7	7	10	11	11
Sprint1	29	42	96	88	94	69
Sprint2	51	100	204	230	186	152
Sprint3	67	100	192	200	189	157

Table 3: Extraneous Cluster Indicator

Subject	ACDC	Bunch	DR	DRB	DRA
Moka	67%	62%	12%	38%	37%
Titan	33%	18%	7%	13%	7%
Minos	29%	29%	20%	18%	18%
Sprint1	24%	30%	13%	22%	20%
Sprint2	23%	32%	12%	19%	22%
Sprint3	30%	36%	19%	21%	27%

Table 4: Lost Information Indicator

Subject	ACDC	Bunch	DR	DRB	DRA
Moka	15(20)	13(20)	5(20)	7(20)	8(20)
Titan	12(13)	7(13)	3(13)	3(13)	3(13)
Minos	5(9)	4(9)	3(9)	2(9)	2(9)
Sprint1	12(29)	4(29)	4(29)	5(29)	9(29)
Sprint2	25(51)	6(51)	6(51)	5(51)	18(51)
Sprint3	37(67)	16(67)	18(67)	11(67)	26(67)

Which method captures most authoritative modules? Table 4 shows that in general ArchDRH family members recognize more authoritative modules than ACDC and Bunch, except that ArchDRH-Re and Bunch have a tie for Sprint1. ACDC failed to recognize more than half of the authoritative modules for all the subject systems.

Table 5: Fragmentation Indicator

Subject	ACDC	Bunch	DR	DRB	DRA
Moka	1.000	1.143	1.467	1.385	1.417
Titan	1.000	1.000	1.300	1.300	1.400
Minos	1.250	1.000	1.333	1.286	1.286
Sprint1	1.882	2.680	3.080	3.042	2.750
Sprint2	2.962	3.089	4.511	3.261	3.576
Sprint3	2.333	2.412	3.327	2.714	2.805

Which method produces most fragments? Table 5 shows that the ArchDRH family always produces more fragments than ACDC and Bunch. The reason is that all ArchDRH family members segregate design rules into separate modules. Our experiment with JHotDraw shows that all the design rules of the system are aggregated into small modules containing one or two elements, which explains why ArchDRH family members always generate more modules and more fragments.

We emphasize E and L values, rather than F because it is more important not to miss important modules, or to produce useless modules. In addition, we already know that the F value will be biased against the ArchDRH family because extracting design rules into separate modules is a feature of ArchDRH, but will unavoidably produce a large F. In the future, we will explore other evaluation methods.

The effect of combining ArchDRH clustering. We compare Bunch vs. ArchDRH-Bunch, ACDC vs. ArchDRH-ACDC to see the effectiveness of such combination. Table 3 shows that ArchDRH-(Bunch/ACDC) generate much fewer extraneous modules than Bunch/ACDC. Table 4 shows that

ArchDRH-Bunch/ACDC always recognize more authoritative modules than Bunch/ACDC. These results demonstrate the advantages of ArchDRH combination.

In addition, comparing ACDC with Bunch, we can see that they always generate drastically different clusterings. Taking Sprint3 as an example, ACDC generates 100 modules, but Bunch generates 193 modules. The number of extraneous modules (30 vs. 69) and unrecognized authoritative modules (37 vs. 16) differ significantly, making it hard for a user to determine which one is better. After combining with ArchDRH clustering, the differences are significantly reduced: for ArchDRH-ACDC and ArchDRH-Bunch, the number of test modules are 157 vs. 189, the numbers of extraneous modules are 42 vs. 40, and the number of unrecognized modules are 26 vs. 11, showing that ArchDRH combination generates more consistent clusterings.

5.3 Qualitative Evaluation

The purpose of our qualitative evaluation is to see if the clusterings produced by the ArchDRH family are useful and meaningful without the existence of authoritative clustering, and to evaluate the effectiveness and utility of ArchDRH-Split, for which we do not have a counterpart to compare with. Due to the different nature of the subject systems and the availability of their domain experts, we discuss the qualitative evaluation results for each subject system separately.

JHotDraw 5.2

We chose this application because it was intentionally designed using multiple design patterns, is well-structured and well-documented.[4] Despite the existence of its documentation with detailed list of design pattern participants, it is not sufficient for us to derive an authoritative clustering for quantitative analysis because a majority of classes in the source code are not pattern participants and we are not sure which classes belong to which modules.

According to its documentation, however, we can evaluate (1) if the top layers in an ArchDRH-based clustering contain design rules that can be mapped to the design rules already detected and documented by other researchers; (2) if the functional modules formed are meaningful. To answer these questions, we applied ArchDRH-Re to the code of JHotDraw 5.2, which produces a DSM with 8 layers with 57 modules.

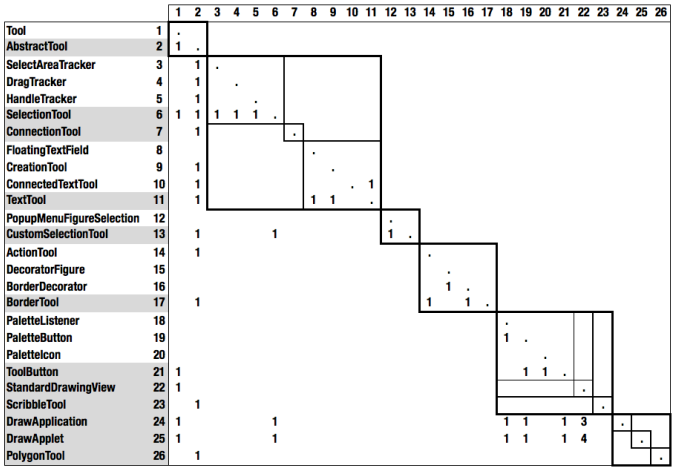
The top two layers of the DSM (not shown for the sake of space) contain design rules, in the form of interfaces, abstract classes, and base classes. Each design rule forms a small module containing only one or two elements. Comparing this DSM with the design pattern documentation of JHotDraw (JHD), we can tell that all the documented classes leading one or more design patterns show up at the two top layers of the DSM.

The rest of the DSM contains 42 function modules within 6 layers. The name assigned to each module hints at its function. We use a sub-DSM extracted by ArchDRH-Split to see if these modules truly reflect meaningful functions. From the description of the JHD documentation, the drawing editor functions are implemented using a strategy pattern with the `Tool` class as the strategy interface. Figure 4 depicts the sub-DSM extracted by ArchDRH-Split, showing all the functional modules that depend on `Tool`.

In this sub-DSM, the rows with shaded background indicate the names of the module assigned by ArchDRH-Re.

[4] `http://java.uom.gr/~nikos/pattern-detection.html`

Figure 4: Partial View of JHotDraw



The first module, which contains `Tool` and `AbstractTool`, dominates (influences) 11 functional modules in lower layers. Some of them are concrete strategies, such as `SelectionTool` (r3-6,c3-6), `ConnectionTool` (r7,c7), `TextTool` (r8-11, c8-11), and some of them are the client of the strategy pattern, such as `DrawApplication` (r24, c24) (a folded module with 14 elements in it). For the sake of space, the last five modules are folded. It is easy to see that each module does reflect one editing function as designed. This DSM also shows that the client of the strategy pattern and the concrete strategy modules are completely separated, as documented by Riehle [22].

This result shows that ArchDRH clustering has the potential to (1) facilitate program understanding by revealing the purposes of a large number of non-pattern related code, complementing design pattern recognition; (2) facilitate software maintenance and evolution. For example, if a new editing function needs to be added, this DSM shows that the new function should implement the `Tool` design rule, and the resulting implementation should add a module to this DSM.

Sprint

As an industrial project with customer obligations, Sprint often sacrifices reliable documentation for meeting deadlines. Using Moka and ArchDRH-Re, we clustered the code of the three releases into three DSMs. Sprint1 DSM has 4 layers and 120 modules, Sprint2 DSM has 6 layers and 315 modules, and Sprint3 DSM has 9 layers and 298 modules.

To ease the examination of these DSMs, we asked our collaborator to provide some design rules they used so that we can split big DSMs into smaller ones. Although there is no documentation and the design decisions used in the first two versions were hard to be retrieved, our collaborator was able to recall the design rules applied to refactor Sprint2 into Sprint3. Using Titan, he was able to identify these design rules, in the form of interfaces and classes easily because they are all aggregated into the top layers of the DSM.

Our collaborator specified five sets of design rules: DR1: `IResultSet`, DR2: `IMetadata` and `IMetadataProvider`, DR3: `Expression`, DR4: `IDataSource`, `IDataProvider` and `IContentStore`, DR5: `IReportBlock`, `IReportAxis` and `CDLAbsReport`. We extracted five smaller DSMs accordingly from

Sprint3 using ArchDRH-Split, each having 19, 46, 34, 32, 42 modules respectively, and following one set of design rules.

After we presented these sub-DSMs to our collaborator, he was able to quickly identify implementation errors by discerning functional modules that should not depend on the given design rules. Concretely, he identified 2 erroneous and 1 questionable (needs to be confirmed) dependencies on DR1, 1 erroneous and 1 questionable dependencies on DR2, 4 erroneous dependencies on DR3, 1 questionable dependency to DR4, and 2 erroneous dependencies on DR5. These erroneous dependences are considered as architectural drift/violation. Our collaborator also identified several modules that are too large or contain multiple functions that should be separated. These functions are unexpectedly coupled by more subtle but suspicious dependencies that require further investigation.

CourseQ&A

CourseQ&A is a web-based information management system for teachers to create and manage exams and for students to take these exams. The system is designed by our academic collaborators and implemented by their former students who were not reachable. The design was modeled using a dataflow diagram, and the code was implemented using Java and JSP pages. The high-level design had 17 business processes with 8 databases. Each process was implemented by one or more Java files. Each process was a relatively independent function, so the expect number of modules was around 17.

We reverse-engineered the code, generated a DSM using ArchDRH-Re, and sent it back to our collaborators. To their surprise, the DSM contained 67 modules, many more than they expected. After having a closer look at the DSM, they realized that the students implemented many small independent utility functions, such as exporting grades and searching for students by name. These utility functions were not the main process of the system, not modeled in the dataflow diagram, and even the designers were not aware of their existence before they saw the DSM. Our collaborators were also able to identify implementation problems, such as the access of a database table from a class that should not have the right to access that table. Currently, they are working on fixing the implementation and completing the design model using the ArchDRH-Re DSM as reference.

6. DISCUSSION

Next, we discuss the threats to validity and related issues.

Threats to Validity

Our results can be sensitive to the authoritative clusterings provided by our collaborators. ArchDRH family members separate design rules into separated modules, leading to a large number of small modules and high fragmentation value, because designers usually do not view design rules as separate modules.

Our results can be sensitive to the quality of the implementation. In our prior work [5], we noticed that if the implementation has many errors, then none of the clustering methods can produce a decomposition exactly the same as the authoritative one. When an implementation significantly deviates from the intended design, we do not expect any clustering method to work effectively. We plan to conduct sensitivity analyses in the future.

Our results can differ if we compare them with clustering methods other than Bunch and ACDC. More advanced clustering methods, such as concern-based clustering [1, 17], may produce better results with more information. We will explore the possibility of combining ArchDRH with other clustering methods. Our result of ArchDRH-Re may also be sensitive to the selection of a stop threshold value. We will conduct more experiments for further evaluation.

Our results can be sensitive to the evaluation method in use. We chose the three structure indicators proposed by Shtern and Tzerpos [25] because they address the known issues of other prevailing evaluation methods, such as the fact that different clustering methods produce drastically different decompositions. Other evaluation methods, such as EdgeSim and MeCl [19], may produce different results.

Related Issues

All the examples and subject systems presented in this paper are developed using object-oriented programming languages, and the design rules we discussed are in the form of interfaces or classes leading design patterns. However, design rules can take many other forms. In a C program, a design rule can be manifested as a key header file. ODBC and SOA messages can also be viewed as design rules.

7. CONCLUSION

In this paper, we contributed an ArchDRH clustering family. Each member of the family first frames the code structure by identifying key design rules and subordinate modules, followed by other clustering techniques. We evaluated the ArchDRH family both quantitatively and qualitatively using in-house, industrial, and open source software systems. Our evaluation shows that our clustering family produces fewer extraneous modules and recognizes more authoritative modules. ArchDRH-Split allowed our industrial collaborators to quickly identify architectural drift caused by certain implementation errors. We believe this work opens new possibilities of evaluating software architecture quality, which we will explore in the near future.

Acknowledgments

We thank our industrial collaborator, Mr. Baelen, for providing the DSMs and authoritative clusterings of his project. We thank Dr. Xin Peng from Fudan University and Dr. Zhenchang Xing from the National University of Singapore for providing CourseQ&A and sharing their opinions on our ArchDRH clustering. We thank Prof. Spiros Mancoridis's SERG group, in particular, Mr. William Mongan, for providing the Bunch tool.

This work was supported in part by the National Science Foundation of the US under grants CCF-0916891, CCF-1065189, CCF-1116980 and DUE-0837665, and the authors from Nanjing University were partially supported by the National Natural Science Foundation of China under grant 61170066.

8. REFERENCES

[1] P. Andritsos and V. Tzerpos. Information-theoretic software clustering. *IEEE Transactions on Software Engineering*, 31(2):150–165, Feb. 2005.

[2] N. Anquetil and T. Lethbridge. Recovering software architecture from the names of source files. *Journal of Software Maintenance*, 11(3):201–221, May 1999.

[3] N. Anquetil and T. C. Lethbridge. Experiments with clustering as a software remodularization method. In *Proc. 6th Working Conference on Reverse Engineering*, pages 235–255, Oct. 1999.

[4] C. Y. Baldwin and K. B. Clark. *Design Rules, Vol. 1: The Power of Modularity*. MIT Press, 2000.

[5] Y. Cai, D. Iannuzzi, and S. Wong. Leveraging design structure matrices in software design education. In *Proc. 24th Conference on Software Engineering Education and Training*, pages 179–188, May 2011.

[6] Y. Cai and K. J. Sullivan. Modularity analysis of logical design models. In *Proc. 21st IEEE/ACM International Conference on Automated Software Engineering*, pages 91–102, Sept. 2006.

[7] S. Ducasse and D. Pollet. Software architecture reconstruction: A process-oriented taxonomy. *IEEE Transactions on Software Engineering*, 35(4):573–591, July 2009.

[8] T. Eisenbarth, R. Koschke, and D. Simon. Derivation of feature component maps by means of concept analysis. In *Proc. 5th European Conference on Software Maintenance and Reengineering*, pages 176–179, Mar. 2001.

[9] E. T. Freeman, E. Robson, B. Bates, and K. Sierra. *Head First Design Patterns*. O'Reilly Media, Oct. 2004.

[10] E. Gamma, R. Helm, R. Johnson, and J. M. Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, Nov. 1994.

[11] Y.-G. Guéhéneuc and G. Antoniol. DeMIMA: A multilayered approach for design pattern identification. *IEEE Transactions on Software Engineering*, 34(5):667–684, May 2008.

[12] M. Harman, S. Swift, and K. Mahdavi. An empirical study of the robustness of two module clustering fitness functions. In *Proc. Genetic and Evolutionary Computation Conference*, pages 1029–1036, June 2005.

[13] D. H. Hutchens and V. R. Basili. System structure analysis: Clustering with data bindings. *IEEE Transactions on Software Engineering*, 11(8):749–757, Aug. 1985.

[14] R. Koschke and T. Eisenbarth. A framework for experimental evaluation of clustering techniques. In *Proc. 8th International Workshop on Program Comprehension*, pages 201–210, June 2000.

[15] J. I. Maletic and N. Valluri. Automatic software clustering via latent semantic analysis. In *Proc. 14th IEEE/ACM International Conference on Automated Software Engineering*, pages 251–254, Oct. 1999.

[16] S. Mancoridis, B. S. Mitchell, Y.-F. Chen, and E. R. Gansner. Bunch: A clustering tool for the recovery and maintenance of software system structures. In *Proc. 15th IEEE International Conference on Software Maintenance*, pages 50–59, Aug. 1999.

[17] O. Maqbool and H. A. Babri. The weighted combined algorithm: A linkage algorithm for software clustering. In *Proc. 8th European Conference on Software Maintenance and Reengineering*, pages 15–24, Mar. 2004.

[18] O. Maqbool and H. A. Babri. Hierarchical clustering for software architecture recovery. *IEEE Transactions on Software Engineering*, 33(11):759–780, Nov. 2007.

[19] B. S. Mitchell and S. Mancoridis. Comparing the cecompositions produced by software clustering algorithms using similarity measurements. In *Proc. IEEE International Conference on Software Maintenance*, pages 744–753, Nov. 2001.

[20] B. S. Mitchell and S. Mancoridis. On the automatic modularization of software systems using the Bunch tool. *IEEE Transactions on Software Engineering*, 32(3):193–208, 2006.

[21] D. L. Parnas. On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12):1053–8, Dec. 1972.

[22] D. Riehle. *Framework Design: A Role Modeling Approach*. PhD thesis, ETH Zurich, 2000.

[23] M. Saeed, O. Maqbool, H. A. Babri, S. Z. Hassan, and S. M. Sarwar. Software clustering techniques and the use of combined algorithm. In *Proc. 7th European Conference on Software Maintenance and Reengineering*, pages 301–306, Mar. 2003.

[24] K. Sartipi, K. Kontogiannis, and F. Mavaddat. Architectural design recovery using data mining techniques. In *Proc. 4th European Conference on Software Maintenance and Reengineering*, pages 129–139, Feb. 2000.

[25] M. Shtern and V. Tzerpos. Refining clustering evaluation using structure indicators. In *Proc. 25th IEEE International Conference on Software Maintenance*, pages 297–305, Sept. 2009.

[26] C. Tjortjis, L. Sinos, and P. J. Layzell. Facilitating program comprehension by mining association rules from source code. In *Proc. 11th International Workshop on Program Comprehension*, pages 125–133, May 2003.

[27] P. Tonella. Concept analysis for module restructuring. *IEEE Transactions on Software Engineering*, 27(4):351–363, Apr. 2001.

[28] P. Tonella and G. Antoniol. Object oriented design pattern inference. In *Proc. IEEE International Conference on Software Maintenance*, pages 230–238, Sept. 1999.

[29] N. Tsantalis, A. Chatzigeorgiou, G. Stephanides, and S. T. Halkidis. Design pattern detection using similarity scoring. *IEEE Transactions on Software Engineering*, 32(11):896–909, Nov. 2006.

[30] V. Tzerpos and R. C. Holt. ACDC: An algorithm for comprehension-driven clustering. In *Proc. 7th Working Conference on Reverse Engineering*, pages 258–267, Nov. 2000.

[31] S. Wong. *On the Interplay of Architecture and Collaboration on Software Evolution and Maintenance*. PhD thesis, Drexel University, Dec. 2010.

[32] S. Wong, Y. Cai, G. Valetto, G. Simeonov, and K. Sethi. Design rule hierarchies and parallelism in software development tasks. In *Proc. 24th IEEE/ACM International Conference on Automated Software Engineering*, pages 197–208, Nov. 2009.

[33] J. Wu, A. E. Hassan, and R. C. Holt. Comparison of clustering algorithms in the context of software evolution. In *Proc. 21st IEEE International Conference on Software Maintenance*, pages 525–535, Sept. 2005.